



Exploración de datos con Pandas

Clase 2 - Estructura DataFrame y acceso a datos tabulares



¿Qué aprenderás en esta sesión?

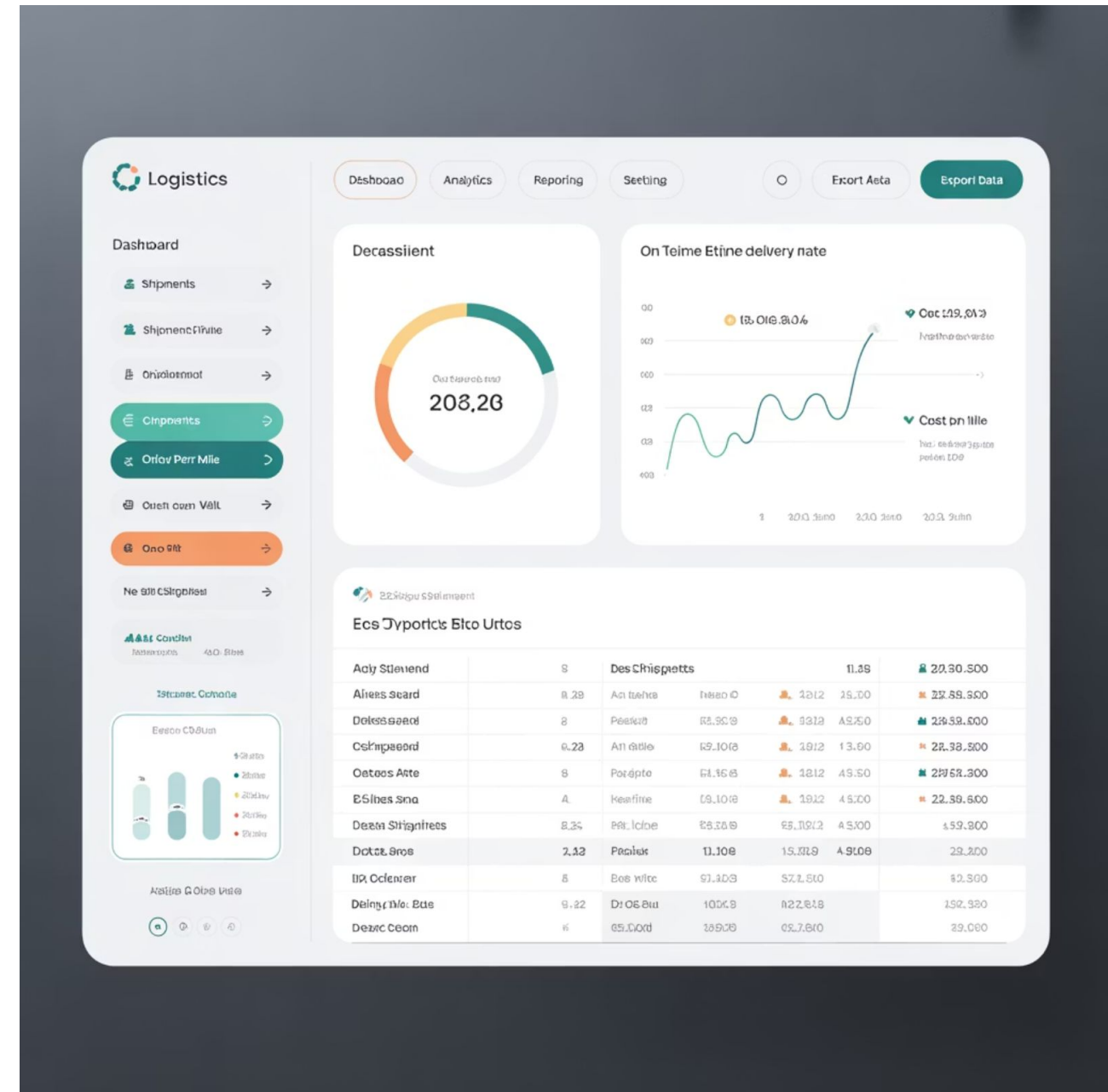
Aprenderemos a trabajar con DataFrames, la estructura de datos tabular más importante en la biblioteca pandas. Exploraremos su creación, visualización, acceso por columnas, filas y condiciones lógicas, para facilitar el análisis de datos bidimensionales de manera eficiente y profesional.

Desafío inicial

Imagina que trabajas en una empresa de logística y recibes diariamente una tabla con los registros de envíos realizados: ciudad, tipo de producto, costo y estado de entrega. Tu jefatura necesita saber si hubo envíos costosos que aún no se han entregado.

¿Cómo podrías cargar, visualizar y filtrar estos datos para entregar esa información rápidamente?

Para responder a este tipo de requerimientos, necesitamos dominar el uso de los DataFrames, estructuras de datos en forma de tabla que permiten realizar este tipo de análisis con unas pocas líneas de código en pandas.



/* ¿Qué es un DataFrame? */

¿Qué es un DataFrame?

Un DataFrame es una estructura de datos bidimensional que permite almacenar y manipular datos en formato tabular, es decir, como si fuese una hoja de cálculo de Excel: con filas y columnas.

Cada columna de un DataFrame es internamente una Serie, y todas las columnas pueden contener diferentes tipos de datos (numéricos, texto, fechas, booleanos). El DataFrame es el núcleo del trabajo con pandas, porque permite realizar:

- Cargas desde archivos CSV, Excel, SQL, JSON, etc.
- Exploración y visualización de información.



2148	3.2.00	0.000	0.000000
2093	9270.00	7.5100	0.000000
1130	29.5808	10.2000	0.000000
9597	85.31.55	20.2.5	0.000000
1929	18.35.48	28.45.3000	0.000000
29.90	39.00.38	91.9.10.560	0.000000
29.50	3.900	72.2.90	0.000000
23.50	3.5.4.50	1.000	0.000000
	7.00.8	2.00	0.000000

Estructura general de un DataFrame

Filas (axis=0)

Cada fila representa un registro o caso.

Columnas (axis=1)

Cada columna representa una variable.

Índice

Identifica de forma única a cada fila (puede ser numérico, de fecha, o cualquier clave relevante).

Ejemplo visual simplificado:

Ciudad	Producto	Costo	Entregado
Santiago	Libros	4500	Sí
Valparaíso	Ropa	6700	No
Concepción	Calzado	8900	Sí

Creación de un DataFrame

¿Cómo se construye un DataFrame en Python con pandas?

Existen múltiples formas de crear un DataFrame, dependiendo de la fuente de datos y la necesidad del análisis. Veamos las más comunes.

Desde un diccionario de listas o arrays

```
import pandas as pd
data = {'Ciudad': ['Santiago', 'Valparaíso', 'Concepción'],
        'Producto': ['Libros', 'Ropa', 'Calzado'],
        'Costo': [4500, 6700, 8900],
        'Entregado': ['Sí', 'No', 'Sí']}
df = pd.DataFrame(data)
print(df)
```


Más formas de crear DataFrames

Desde una lista de diccionarios
(cada dict es una fila)

```
registros = [{'Ciudad':  
'Temuco', 'Producto': 'Ropa',  
'Costo': 7200, 'Entregado':  
'Sí'},  
             {'Ciudad':  
'Iquique', 'Producto':  
'Libros', 'Costo': 5300,  
'Entregado': 'No'}]  
df = pd.DataFrame(registros)
```

Desde archivos externos (más
adelante se profundizará)

```
df = pd.read_csv('datos.csv')
```

Personalizando el índice

```
df = pd.DataFrame(data,  
index=['Pedido1', 'Pedido2',  
       'Pedido3'])
```

La flexibilidad del constructor de `pandas.DataFrame()` permite adaptar la estructura a diversas fuentes y formatos, desde sensores hasta bases de datos. Una vez creado el `DataFrame`, lo siguiente es aprender a acceder a sus componentes.

Selección de columnas o filas

¿Cómo accedemos a partes específicas del DataFrame?

Para manipular y explorar los datos, necesitamos saber cómo acceder a las columnas (variables) o filas (observaciones). Existen diferentes formas.

Selección de columnas

- Acceso por nombre (forma preferida):

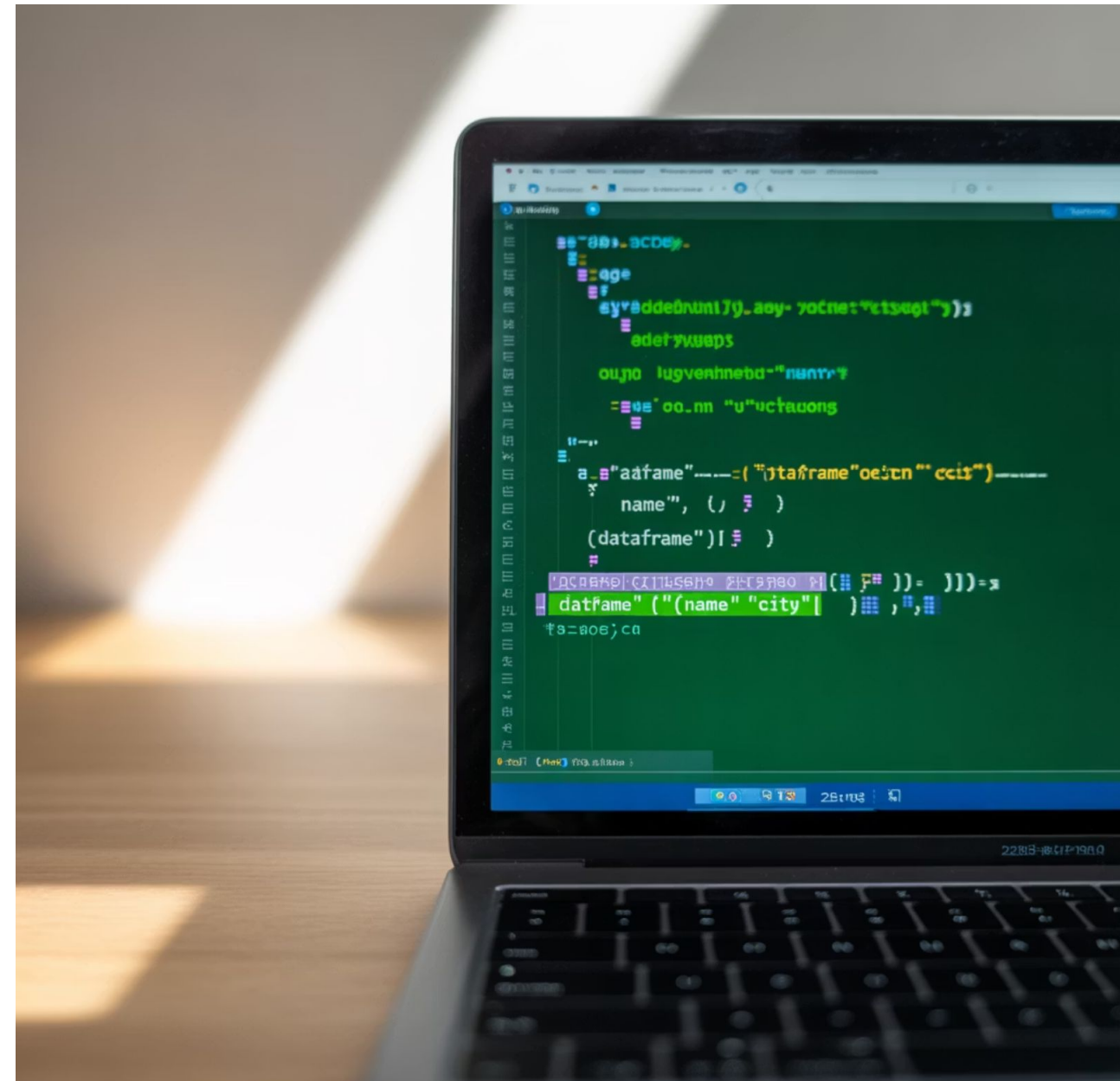
```
df['Ciudad']
```

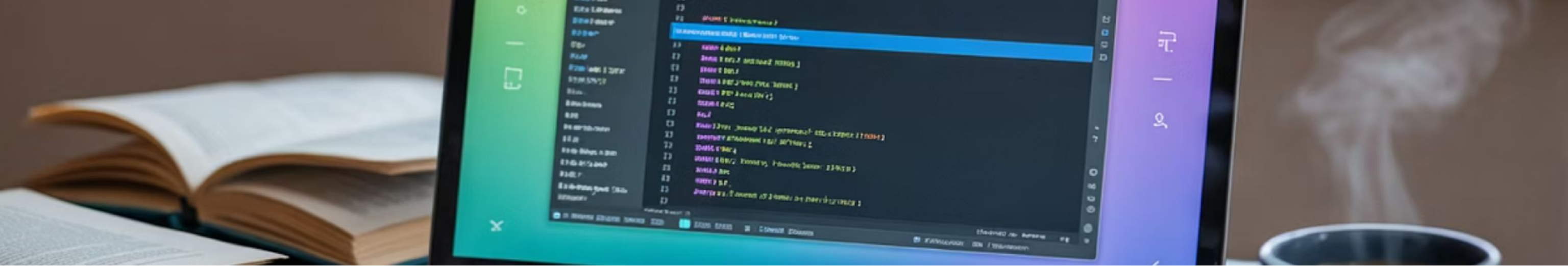
- También puede usarse la notación punto (aunque no se recomienda para nombres con espacios):

```
df.Ciudad
```

- Para seleccionar múltiples columnas:

```
df[['Ciudad', 'Costo']]
```





Selección de filas

Se pueden seleccionar por posición o por índice.

Por posición (iloc):

```
df.iloc[0] # Primera fila
```

Por etiqueta del índice (loc):

```
df.loc['Pedido2']
```

Varias filas por rango:

```
df[1:3]
```

Estos accesos nos permiten construir análisis personalizados y exploraciones selectivas del contenido del DataFrame. Pero para obtener elementos concretos, necesitamos combinar filas y columnas.

4. Selección de elementos de un DataFrame

¿Cómo accedemos a un dato específico dentro del DataFrame?

Para seleccionar una celda o subconjunto específico de datos, combinamos el acceso a filas y columnas usando `.loc[]` o `.iloc[]`.

Acceso por etiqueta (loc)

```
df.loc['Pedido1', 'Costo']
```

Este tipo de acceso es muy útil para:

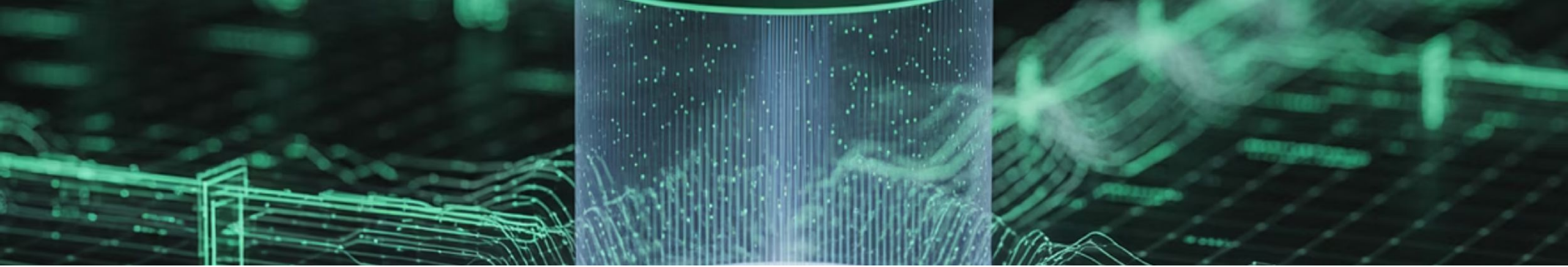
- Generar reportes personalizados.
- Visualizar solo ciertas variables.
- Preparar subconjuntos para análisis o exportación.

Acceso por posición (iloc):

```
df.iloc[0, 2]  
# primera fila, tercera columna
```

También es posible seleccionar subtablas:

```
df.loc[['Pedido1', 'Pedido3'], ['Ciudad', 'Costo']]
```



Preparándonos para el filtrado condicional

Ya sabemos acceder a celdas y subconjuntos.

Pero, ¿cómo filtramos datos según una condición como "ventas mayores a \$100.000" o "empleados mayores de 30 años"?

A continuación lo exploramos.

5. Selección condicional en un DataFrame

¿Cómo filtramos datos según condiciones lógicas?

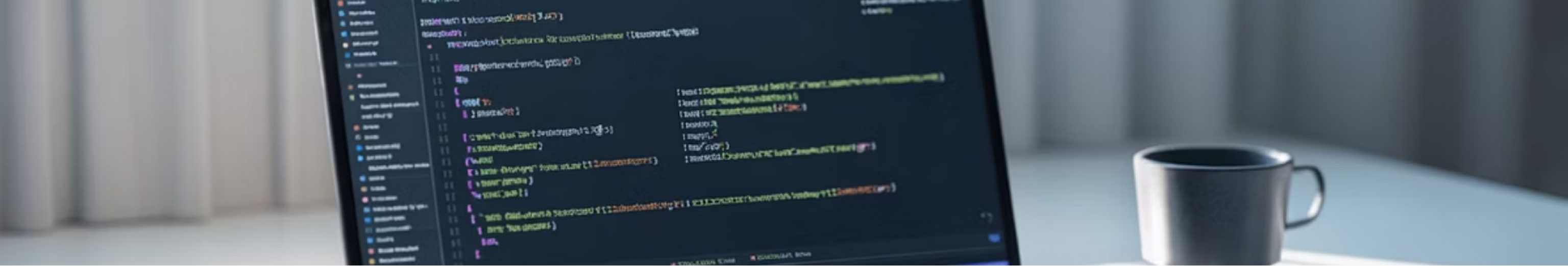
Uno de los aspectos más poderosos de pandas es la posibilidad de aplicar condiciones booleanas para filtrar registros que cumplen criterios específicos.

Ejemplo: mostrar pedidos con costo mayor a \$6000

```
df[df['Costo'] > 6000]
```



Ciudad	Producto	Costo	Entregado
Valparaíso	Ropa	6700	No
Concepción	Calzado	8900	Sí



Filtrar por múltiples condiciones

```
df[(df['Costo'] > 6000) & (df['Entregado'] == 'No')]
```

Operador &

El operador & representa "y" lógico.

Operador |

El operador | representa "o" lógico.

Paréntesis

Cada condición debe ir entre paréntesis.

Este tipo de operaciones permite obtener con rapidez subconjuntos de datos relevantes, sin bucles ni estructuras complejas.

Ejercicio - Filtrando información crítica desde una tabla de entregas

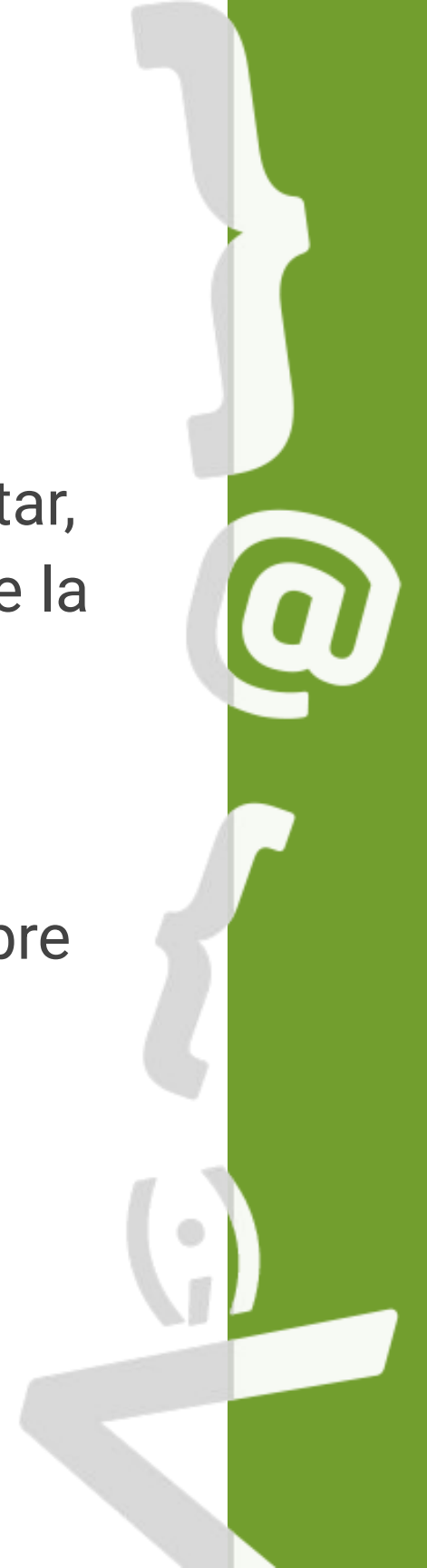


Ejercicio

Instrucciones

Objetivo: Aplicar la estructura DataFrame de la biblioteca pandas para representar, explorar y filtrar información relevante de un conjunto de datos tabular, mediante la selección de columnas, filas, celdas y condiciones lógicas.

Situación problema: Trabajas en una empresa de distribución de productos electrónicos. Recientemente, la jefatura de operaciones necesita un informe sobre los pedidos que aún no han sido entregados y cuyo valor es superior a los \$50.000. Tu misión es identificar estos pedidos a partir de una tabla de datos operativos y reportarlos de forma clara, utilizando herramientas de pandas.



Ejercicio

Instrucciones

Crear el DataFrame base con los datos siguientes:

```
import pandas as pd
data = {'ID Pedido': [1001, 1002, 1003, 1004, 1005],
        'Cliente': ['Juan', 'Sofía', 'Camila', 'Ignacio', 'Fernanda'],
        'Valor ($)': [62000, 48000, 71000, 32000, 90000],
        'Entregado': ['No', 'Sí', 'No', 'Sí', 'No']}
df = pd.DataFrame(data)
df.index = ['P1', 'P2', 'P3', 'P4', 'P5'] # Etiquetas personalizadas para filas
```


Ejercicio

Instrucciones

Visualizar la tabla completa:

```
print(df)
```

Seleccionar solo la columna Cliente:

```
df['Cliente']
```

Mostrar la fila con índice 'P3':

```
df.loc['P3']
```

Obtener el valor del pedido P5 en la columna Valor (\$):

```
df.loc['P5', 'Valor ($)']
```

ID Pedido	Cliente	Valor (\$)	Entregado
1001	Juan	62000	No
1002	Sofía	48000	Sí
1003	Camila	71000	No

Ejercicio

Instrucciones

Filtrar pedidos con valor mayor a \$50.000 y que no han sido entregados:

```
filtro = (df['Valor ($)'] > 50000) & (df['Entregado'] == 'No')  
df[filtro]
```

Preguntas de reflexión y cierre

- ¿Cómo cambia tu análisis si no aplicas condiciones múltiples con &?
- ¿Qué ventajas observas al acceder por nombre (loc) en lugar de por posición (iloc)?
- ¿Qué precauciones tomarías si este DataFrame fuera mucho más grande?

Ejercicio - Selección de columnas, filas y filtrado condicional

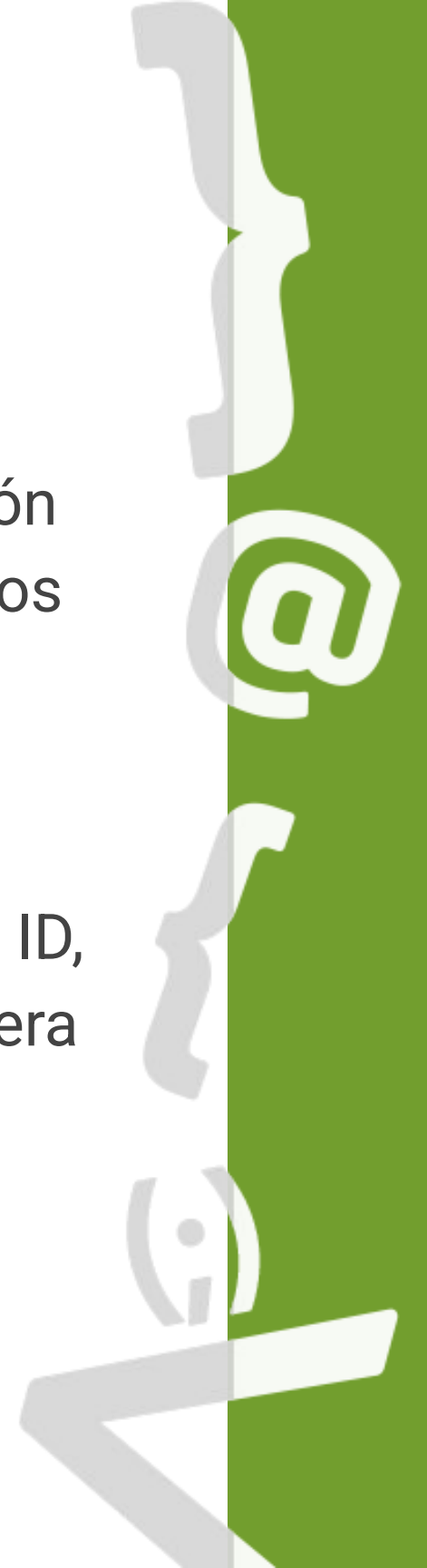


Ejercicio

Instrucciones

Objetivo: Reforzar la exploración de DataFrames mediante ejercicios de selección de columnas, filas y filtrado condicional, replicando el análisis de datos operativos en un entorno de trabajo simulado.

Contexto: Trabajas en el área de soporte de una empresa que lleva el control de incidentes técnicos registrados en distintas sucursales. Cada incidente tiene un ID, una prioridad, una sucursal asociada y un estado (resuelto o pendiente). Se espera que identifiques rápidamente los casos críticos que aún no han sido resueltos.



Ejercicio

Instrucciones

Crea el siguiente DataFrame:

```
data = {'ID Incidente': [2001, 2002, 2003, 2004, 2005], 'Sucursal': ['Valparaíso', 'Santiago', 'La  
Serena', 'Temuco', 'Antofagasta'], 'Prioridad': ['Alta', 'Media', 'Alta', 'Baja', 'Alta'], 'Estado':  
['Pendiente', 'Resuelto', 'Pendiente', 'Resuelto', 'Pendiente']}  
df = pd.DataFrame(data)
```

Ejercicio

Instrucciones

1

Muestra el listado completo de incidentes.

2

Selecciona todas las sucursales afectadas por incidentes de prioridad "Alta".

3

Filtra los registros en los que:

- La prioridad sea "Alta".
- El estado sea "Pendiente".

4

¿Cuántos incidentes críticos sin resolver hay actualmente?

5

Extrae los valores de Sucursal correspondientes solo a los casos filtrados anteriormente.

Resumen



Estructura de DataFrame

Comprendimos la estructura de DataFrame y su utilidad profesional para representar información tabular.



Acceso a datos

Accedimos a columnas, filas, celdas y subconjuntos usando `iloc` y `loc`.



Creación de DataFrames

Aprendimos a crear DataFrames desde estructuras internas y externas.

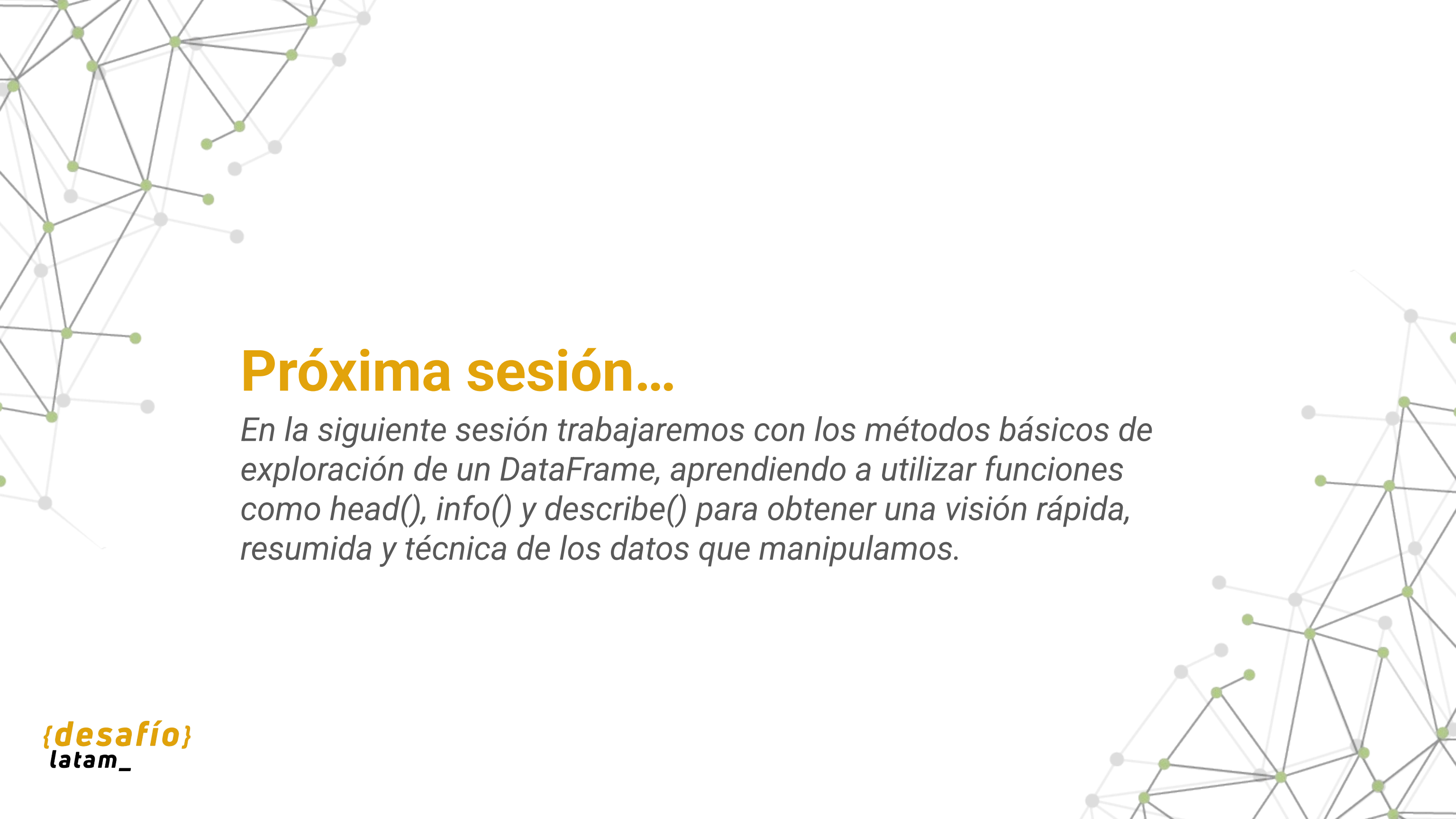


Filtrado condicional

Aplicamos filtros lógicos para obtener información específica a partir de condiciones múltiples.

¿Cómo seleccionas una columna
específica de un DataFrame?
¿Cómo accederías al valor de la
tercera fila en la columna
"Costo"?





Próxima sesión...

En la siguiente sesión trabajaremos con los métodos básicos de exploración de un DataFrame, aprendiendo a utilizar funciones como `head()`, `info()` y `describe()` para obtener una visión rápida, resumida y técnica de los datos que manipulamos.